

Laboratorio 5

Programmazione di Sistema | A.A. 2025–26 | 20 e 22 Maggio 2026

1 Esercizio 1 – Lista concatenata

1.1 Descrizione

Il file `es0501.rs` contiene tre moduli: `mem_inspect`, `List1`, `List2` e `dlist`. I primi due implementano una lista singolarmente concatenata con due approcci diversi, il terzo una lista doppiamente concatenata.

Il modulo `mem_inspect` fornisce le funzioni `dump_object` e `dump_memory` per ispezionare oggetti e smart pointer in memoria. Va usato per rispondere alle domande di analisi.

1.2 Sviluppo

Passo 1 – List1: lista con enum

Il primo modello usa una enum ricorsiva:

```
pub enum Node<T> {  
    Cons(T, Box<Node<T>>),  
    Nil,  
}
```

Implementare i metodi `new`, `push`, `pop`, `peek` e `take` come descritto nei commenti del file.

Attenzione

In `push` non è possibile muovere `self.head` direttamente perché lascerebbe la struct in uno stato non definito. Usare `std::mem::replace` per estrarre il valore e sostituirlo atomicamente con `Nil`.

Passo 2 – List2: lista con struct

Il secondo modello usa una struct con un campo `next` opzionale:

```
pub struct Node<T> {  
    elem: T,  
    next: Option<Box<Node<T>>>,  
}
```

Implementare gli stessi metodi di `List1`.

Nota

`Option<T>` ha il metodo `take()` che estrae il valore e sostituisce l'opzione con `None` equivalente a `mem::replace` ma più idiomatico quando si lavora con `Option`.

Passo 3 - Ispezione della memoria

Una volta implementate entrambe le liste, usare `dump_object` per ispezionare i nodi:

```
let mut l1 = List1::List::<i32>::new();
l1.push(10);
let mut l2 = List2::List::<i32>::new();
l2.push(10);
```

Domanda

Quale problema si incontra usando `Box` per i puntatori `prev` e `next` in una lista doppiamente concatenata? Come si risolve? Quali smart pointer servono e perché?

Passo 4 - ListIter: iteratore sulla lista

Implementare la struct `ListIter` e il trait `Iterator` per poter iterare sui valori della lista con un ciclo `for`. Decommentare il metodo `iter()` in `List2`.

Nota

Quando si fa `match` su un `Option` per riferimento, il pattern estrae un riferimento al valore interno:

```
let a = Some(5);
let b = &a;
match b { Some(i) => ... } // i e' &i32, non i32
```

Passo 5 - dlist: lista doppiamente concatenata

Modificare `List2` per creare una lista doppiamente concatenata con puntatori sia alla testa che alla coda, e ogni nodo che punta anche al predecessore.

Implementare:

- `push_front` e `pop_front` – inserimento e rimozione in testa
- `push_back` e `pop_back` – inserimento e rimozione in coda
- `popn(&mut self, n: usize) -> Option<T>` – rimozione dell'elemento in posizione `n`

Domanda

Quale problema si incontra nel definire la struttura dei nodi dell'albero? Quali smart pointer sceglieresti e perché?

Domanda

In `pop` si vuole restituire il valore estratto dal nodo rimosso. Perché non è sempre possibile estrarre il valore da un `Rc`? Quando è possibile farlo con `Rc::try_unwrap`?

2 Esercizio 2 – File system in memoria

2.1 Descrizione

Realizzare una struct `FileSystem` che implementi in memoria la struttura di un file system, replicando qualsiasi directory e il suo contenuto presente su disco.

Il file system è un albero composto da tre tipi di elementi:

```
enum FSItem {
    Directory(Directory), // nome, figli, metadati, padre
    File(File),           // nome, metadati, padre
    SymLink(Link),        // path a cui punta, padre
}
```

Nota

I link simbolici possono puntare a path non validi – gestire questo caso senza panic. Aggiungere il supporto ai link solo in una seconda fase, dopo aver fatto funzionare file e directory.

2.2 Path

Ogni metodo accetta path assoluti e relativi:

- **assoluti** – iniziano con `/` e partono dalla radice
- **relativi** – partono dalla directory corrente; `.` indica la directory corrente, `..` indica il padre

2.3 Sviluppo

Passo 1 – Struttura dati

Definire le struct `Directory`, `File` e `Link` con i campi necessari. Ogni nodo deve poter risalire al padre oltre che scendere verso i figli.

Domanda

Quale smart pointer permette di avere più riferimenti allo stesso nodo? Perché serve `RefCell` oltre a `Rc`? Quali puntatori devono essere `Rc` e quali `Weak` per evitare cicli?

Passo 2 – Implementare i metodi

```
impl FileSystem {
    pub fn new() -> Self
    pub fn from_disk(path: &str) -> Result<Self, ...>
    pub fn change_dir(&mut self, path: String) -> Result<...>
    pub fn make_dir(&mut self, path: String, name: String) -> Result<...>
    pub fn make_file(&mut self, path: String, name: String) -> Result<...>
    pub fn rename(&self, path: String, new_name: String) -> Result<...>
    pub fn delete(&self, path: String) -> Result<...>
    pub fn find(&self, path: String) -> Result<...>
}
```

Domanda

Alcuni metodi come `rename` e `delete` modificano la struttura ma ricevono `&self` invece di `&mut self`. Come è possibile?

Passo 3 - Ispezione della memoria

Usare `dump_object` e `dump_memory` per disegnare il layout degli smart pointer usati per i nodi.

Domanda

Qual è la dimensione di un `Rc<RefCell<FSItem>`? Cosa contiene in memoria un `Rc` quanti puntatori, dove sono i reference count?

Bonus (facoltativo)

- Aggiungere il supporto ai link simbolici, gestendo i path non validi.
- Implementare un metodo `print_tree` che stampa la struttura del file system in modo simile al comando `tree` di Unix.